

Parallel Incremental Maintenance of TPC-H Query 3 with Apache Flink

YUN Hanxu

Student ID: 21286712

MSc in Information Technology

CSIT6910 Independent Project

Supervisor: Prof. Ke Yi

Code: <https://github.com/silverpioup/Self-Project>

Abstract

This project implements continuous incremental maintenance of TPC-H Query 3 using Apache Flink. Instead of recomputing the query after every update, the system stores relation indexes and result-group revenue in Flink managed keyed state. Orders and lineitems are partitioned by order key, while customer updates are replicated to the order shards. The stateful operator therefore runs with actual parallelism 1, 2, 4, or 8. Complete Flink snapshots are compared with an independent SQLite execution. All checked snapshots pass on a corner-case stream, a 155,400-update synthetic stream, and TPC-H scale-factor 0.01 data.

1. Background and Related Work

Continuous analytical applications evaluate registered queries while their input relations change. Re-executing a multi-table query after every insertion or deletion repeats joins and aggregation work unrelated to the changed record. Incremental view maintenance instead derives the answer change from the update and indexed state.

Cquirrel studies continuous selection, projection, join, and aggregation queries over acyclic relational schemas. The AJU work supplies the fuller algorithmic basis for maintaining acyclic foreign-key joins under updates. TPC-H Query 3 matches this setting through the chain customer, orders, and lineitem.

2. Query and Algorithm

Query 3 selects BUILDING-segment customers, orders placed before 15 March 1995, and lineitems shipped after that date. It groups by order key, order date, and ship priority, and sums extended price multiplied by one minus discount.

```
SELECT l_orderkey,  
SUM(l_extendedprice * (1 - l_discount)) AS revenue,  
o_orderdate, o_shippriority  
FROM customer, orders, lineitem  
WHERE c_mktsegment = 'BUILDING'  
AND c_custkey = o_custkey AND l_orderkey = o_orderkey  
AND o_orderdate < DATE '1995-03-15'  
AND l_shipdate > DATE '1995-03-15'  
GROUP BY l_orderkey, o_orderdate, o_shippriority  
ORDER BY revenue DESC, o_orderdate LIMIT 10;
```

A qualifying lineitem contributes $extendedprice \times (1 - discount)$. For a lineitem update, the algorithm checks one order and customer. For an order update, only lineitems indexed under that order are visited. For a customer update, only orders indexed under the customer and their lineitems are visited. Changes caused by one input are combined by group. Non-zero results emit UPSERT_GROUP; groups reduced to zero emit DELETE_GROUP.

3. Implementation Details

The implementation uses Java, Flink 1.19.1, Maven, and JDK 17. A single ordered file source assigns global sequence numbers. Orders and lineitems are routed to orderkey modulo the requested parallelism. Customer updates are copied to every shard because one customer may have orders in several partitions. A keyed process function then runs at parallelism 1, 2, 4, or 8.

Each shard owns Flink MapState for customers, orders, orders by customer, lineitems by order, and current group revenue. All records of one order share a state partition, while every shard has the customer state needed for its orders. The repository also contains the TPC-H converter, deterministic generators, automated SQLite comparison, benchmark runners, result tables, and reproducible build commands.

4. Experimental Setup and Data Sets

Experiments were run locally with Flink 1.19.1 and JDK 17. Bounded-file wall time includes Flink startup and shutdown. Throughput is input updates divided by wall time. Latency is measured from sequence assignment to completion of keyed-state maintenance and reported as mean, P50, P95, and P99 in microseconds.

Data set	Description	Updates
Sample	Hand-written insert/delete corner cases	13
Synthetic large	Update-heavy three-relation stream	155,400
TPC-H SF 0.01	1,500 customers; 15,000 orders; 60,175 lineitems	76,675

TPC-H tables were generated with DuckDB's TPC-H-compatible dbgen extension and converted into the update format. Correctness was evaluated independently by reconstructing complete Flink state from deltas and comparing every group with SQLite. The sample passed 13 snapshots at each parallelism 1, 2, 4, and 8. The two large inputs each passed 16 complete snapshots at parallelism 8.

5. Experimental Results and Discussion

P	Updates/s	Mean us	P50 us	P95 us	P99 us
1	37,788.28	14,632.15	14,514	22,875	23,374
2	37,935.63	17,795.53	16,015	29,085	29,857
4	37,715.33	5,963.91	2,961	18,259	21,590
8	37,663.54	6,338.67	3,969	19,328	27,359

TPC-H SF 0.01 throughput remains near 37,700–37,900 updates per second because the ordered source, startup cost, and customer replication limit scaling. P50 latency falls from 14,514 us at parallelism 1 to 2,961 us at parallelism 4, showing concurrent order-shard processing. Parallelism 8 adds replication overhead and does not improve on parallelism 4. The synthetic workload similarly shows that frequent customer updates make broadcast cost visible; no linear speedup is claimed.

6. Discussion

The project meets the central functional objectives: three-table TPC-H processing, insertions and deletions, paper-inspired incremental maintenance, managed Flink state, changed-group output, independent full-state verification, larger data sets, actual stateful-operator parallelism, and percentile latency measurement.

The implementation is query-specific rather than a general Cquirrel compiler. Customer updates are replicated to all shards, which is expensive when customer changes are frequent. The measured scale-factor data was generated with a compatible TPC-H extension; the results are local project measurements rather than audited TPC results.

7. Future Directions

Future work could compile additional SPJA queries and derive state indexes automatically. Customer replication could be reduced through a customer-to-shard subscription index or a separate enrichment stage. A long-running Kafka and cluster evaluation should measure steady-state throughput, checkpoint overhead, failure recovery, state-backend choices, and larger official DBGEN scale factors.

8. Conclusion

The final system maintains TPC-H Q3 as a stream of local result changes rather than recomputing the query. Managed keyed state and order sharding preserve correctness up to parallelism eight in the tested environment. Automated complete-state comparison and reproducible experiments provide evidence for both correctness and performance.

References

1. Q. Wang et al., "Cquirrel: Continuous Query Processing over Acyclic Relational Schemas," PVLDB 14(12), 2021.
2. Q. Wang and K. Yi, "Maintaining Acyclic Foreign-Key Joins under Updates," SIGMOD, 2020.
3. Transaction Processing Performance Council, TPC Benchmark H Standard Specification, Revision 3.0.1.
4. Apache Flink 1.19 Documentation.
5. Project code: <https://github.com/silverpioup/Self-Project>.

Appendix note. The fourth meeting is scheduled for early August. Its entry is a draft agenda and should be changed to completed minutes after the meeting.

Minutes of the 1st Project Meeting

Date	Wednesday, June 10, 2026
Time	4:00 pm-4:22 pm
Place	Online meeting
Present	Prof. Ke Yi; YUN Hanxu; other CSIT6910 students
Apology	None recorded
Note-taker	YUN Hanxu

1. Approval of minutes

This was the first project meeting, so there were no previous minutes to approve.

2. Discussion items

The supervisor introduced TPC-H and the two project directions. For the Flink project, the query should involve at least three tables; Q3 was identified as the standard minimum example. The student should generate an insertion/deletion stream, implement incremental processing with a Flink process function and maintained state, output affected groups, verify snapshots against a conventional database, and then evaluate throughput and parallelism. Cquirrel was assigned as the primary paper, with AJU as the fuller reference.

3. Meeting adjournment and next meeting

The briefing ended at approximately 4:22 pm. The next consultation would review the selected query, update format, and implementation design.

Minutes of the 2nd Project Meeting

Date	Wednesday, July 1, 2026
Time	4:00 pm-4:30 pm
Place	Online meeting
Present	Prof. Ke Yi; YUN Hanxu
Apology	None recorded
Note-taker	YUN Hanxu

1. Approval of minutes

The first meeting's requirements and the selection of TPC-H Q3 were reviewed.

2. Discussion items

The Q3 predicates, foreign-key chain, group key, and revenue expression were confirmed. The update representation was defined for customer, orders, and lineitem insertions and deletions. The planned state consisted of primary-key maps, reverse indexes from customer to orders and order to lineitems, and a revenue map. SQLite was selected as the independent correctness baseline.

3. Meeting adjournment and next meeting

The next meeting would review the runnable Flink implementation and sample correctness results.

Minutes of the 3rd Project Meeting

Date	Friday, July 24, 2026
Time	4:00 pm-4:30 pm
Place	Online meeting
Present	Prof. Ke Yi; YUN Hanxu
Apology	None recorded
Note-taker	YUN Hanxu

1. Approval of minutes

The query definition, update format, state indexes, and SQLite verification plan from the second meeting were reviewed.

2. Discussion items

The working Java/Flink job and sample delta output were reviewed. Insertions, deletions, replacement operations, and removal of zero-revenue groups were tested. The correctness process was strengthened to reconstruct complete Flink state and compare every result group with SQLite. The next tasks were to convert scale-factor data, move the state into Flink managed keyed state, and test parallelism 1, 2, 4, and 8.

3. Meeting adjournment and next meeting

The next meeting would review larger-data correctness, parallel-performance measurements, and the final report.

Minutes of the 4th (Scheduled Draft) Project Meeting

Date	Monday, August 3, 2026
Time	4:00 pm-4:30 pm
Place	Online meeting
Present	Prof. Ke Yi; YUN Hanxu
Apology	None recorded
Note-taker	YUN Hanxu

1. Approval of minutes

The implementation and verification actions recorded in the third meeting will be reviewed.

2. Discussion items

The final keyed-state design, TPC-H table conversion, and experimental results will be presented. The sample passed at parallelism 1, 2, 4, and 8. The 155,400-update synthetic stream and 76,675-update TPC-H SF 0.01 stream each passed 16 complete SQLite snapshots at parallelism 8. Throughput and mean/P50/P95/P99 latency will be discussed, including the cost of replicating customer updates. The report structure, repository link, limitations, and future directions will be reviewed.

3. Meeting adjournment and next meeting

The intended outcome is approval of the final report and code submission, subject to any corrections requested during the meeting.